

Módulo 16: Arquitectura Limpia

Contenidos clave

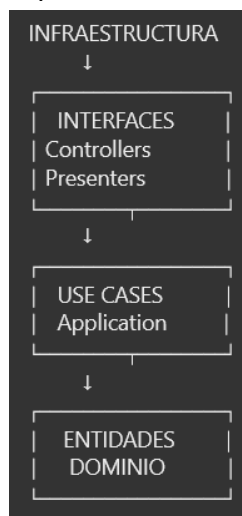
Arquitectura Limpia: Las reglas de negocio deben ser independientes de frameworks, bases de datos, APIs y otras tecnologías externas.

Entidades, casos de uso, controladores/presenters/gateways:

- Entidades: representan conceptos importantes del negocio y contienen las reglas que deben mantenerse siempre.
- Casos de uso: representan acciones que el sistema puede realizar. Un caso de uso coordina entidades y dependencias.
- Controllers: recibe la petición externa y la transforma en una llamada al caso de uso. "Recibo la petición".
- Presenters: transforma el resultado del caso de uso en un formato que pueda consumir el exterior. "Preparo la respuesta".
- Gateways: es una abstracción para comunicarse con algo externo.

Reglas de dependencia y separación de capas:

- Regla de dependencia: de las cosas más importantes en Clean Architecture. Las dependencias deben apuntar hacia el centro.



Las capas externas pueden conocer las internas; las internas no deben conocer las externas.

Unit of Work y eventos de dominio:

- UoW - Unit of Work: representa una unidad de trabajo/transacción. "Todo esto forma parte de una misma operación; o se guarda todo, o no se guarda nada".
- Eventos de dominio: representa algo que ya ocurrió en el negocio.
- Evento vs llamada directa: sin eventos el caso de uso termina sabiendo demasiadas cosas, con eventos se permite desacoplar funcionalidades.

Estrategias de migración hacia arquitectura limpia:

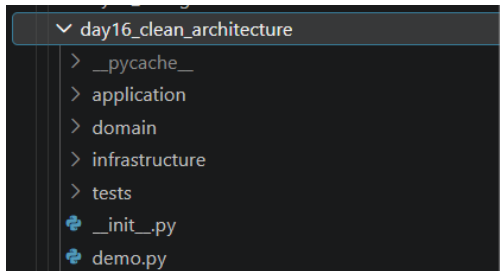
1. Identificar la lógica de negocio (separar por capas).
2. Extraer entidades.
3. Extraer casos de uso.
4. Crear interfaces/gateways.

5. Mover implementaciones a infraestructura.
6. Agregar UoW y eventos (para migrar progresivamente).

Objetivos

- Estructurar un servicio con capas independientes y reglas claras.

La estructura se separó en capas:



- Gestionar transacciones y publicar eventos.

El contrato se definió en [ports.py](#), en la clase UnitOfWork, el adaptador en memoria implementa el commit y rollback, el adaptador utiliza la sesión real. El evento está definido en [events.py](#) y se publica después del commit lo que evita publicarlo si la persistencia falla.

Laboratorio

- Reestructurar Orders a arquitectura limpia

- Capa de dominio:

Contiene las entidades y reglas de negocio.

```
5 @dataclass(frozen=True)
6 class OrderItem:
7     product_name: str
8     quantity: int
9     unit_price: Decimal
10
11     @property
12     def subtotal(self) -> Decimal:
13         return self.unit_price * self.quantity
```

La orden calcula su total dentro del dominio:

```
16 @dataclass
17 class Order:
18     id: int
19     customer_id: int
20     items: tuple[OrderItem, ...]
21     status: str = "PENDING"
22
23     @property
24     def total(self) -> Decimal:
25         return sum(
26             (item.subtotal for item in self.items),
27             Decimal("0.00"),
28         )
29
```

Esta lógica no depende de SQLAlchemy, ni de bases de datos, HTTP, FastAPI o servicios externos.

- Capa de aplicación:

Aquí se implementó el caso de uso CreateOrder:

```
15 class CreateOrder:
16     def __init__(
17         self,
18         unit_of_work: UnitOfWork,
19         event_publisher: EventPublisher,
20     ) -> None:
21         self._unit_of_work = unit_of_work
22         self._event_publisher = event_publisher
23
```

El caso de uso coordina la validación de la entrada, la creación de la entidad, la persistencia, el commit, la publicación del evento, y la presentación de la respuesta.

```
31         with self._unit_of_work as uow:
32             order = Order(
33                 id=uow.orders.next_id(),
34                 customer_id=data.customer_id,
35                 items=data.items,
36             )
37
38             uow.orders.save(order)
39             uow.commit()
40
41             self._event_publisher.publish(
42                 OrderCreated(
43                     order_id=order.id,
44                     customer_id=order.customer_id,
45                     total=order.total,
46                 )
47             )
```

- Capa de infraestructura:

Aquí están las implementaciones concretas: el repositorio en memoria, Unit of Work en memoria, repositorio SQLAlchemy y el publicador de eventos en memoria.

La aplicación no depende directamente de estas clases; las recibe mediante los puertos.

- Introducir UoW y Presenter

- Presenter:

[presenters.py](#):

```
7 class OrderResponse(TypedDict):
8     id: int
9     customer_id: int
10    total: Decimal
11    status: str
12
13
14 def present_order(order: Order) -> OrderResponse:
15     return {
16         "id": order.id,
17         "customer_id": order.customer_id,
18         "total": order.total,
19         "status": order.status,
20     }
```

La función `present_order` transforma la entidad interna en una respuesta estable para el exterior.

- Unit of Work:

[ports.py](#):

```
22 class UnitOfWork(Protocol):
23     orders: OrderRepository
24
25     def __enter__(self) -> "UnitOfWork": ...
26
27     def __exit__(
28         self,
29         exc_type: type[BaseException] | None,
30         exc_value: BaseException | None,
31         traceback: object | None,
32     ) -> None: ...
33
34     def commit(self) -> None: ...
35
36     def rollback(self) -> None: ...
37
```

El adaptador en memoria implementa el `commit` y el `rollback`.

- Evento `OrderCreated` manejado en aplicación.

[events.py](#):

```
4
5 @dataclass(frozen=True)
6 class OrderCreated:
7     order_id: int
8     customer_id: int
9     total: Decimal
10
```

Katherine Quiryat Rosales Garduño

Los eventos se publican después de confirmar la transacción.

La arquitectura permite sustituir adaptadores sin cambiar el caso de uso.